



UNIVERSITY OF CALOOCAN CITY
COMPUTER ENGINEERING DEPARTMENT



Data Structure and Algorithm

Laboratory Activity No. 14

Tree Structure Analysis

Submitted by:
Gabijan, Rhovic M.

Instructor:
Engr. Maria Rizette H. Sayo

November 10, 2025

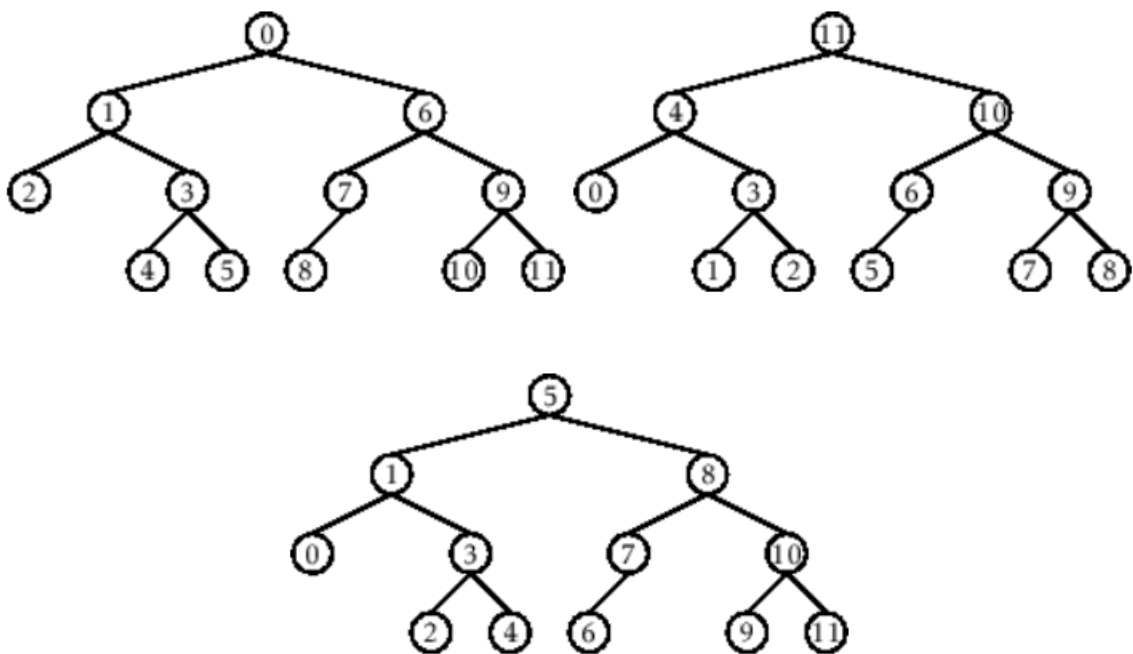
I. Objectives

Introduction

An abstract non-linear data type with a hierarchy-based structure is a tree. It is made up of links connecting nodes (where the data is kept). The root node of a tree data structure is where all other nodes and subtrees are connected to the root.

This laboratory activity aims to implement the principles and techniques in:

- To introduce Tree as Non-linear data structure
- To implement pre-order, in-order, and post-order of a binary tree



- Figure 1. Pre-order, In-order, and Post-order numberings of a binary tree

II. Methods

- Copy and run the Python source codes.
- If there is an algorithm error/s, debug the source codes.
- Save these source codes to your GitHub.
- Show the output

1. Tree Implementation

```
class TreeNode:
    def init (self, value):
        self.value = value
        self.children = []

    def add_child(self, child_node):
        self.children.append(child_node)

    def remove_child(self, child_node):
        self.children = [child for child in self.children if child != child_node]
```

```

def traverse(self):
    nodes = [self]
    while nodes:
        current_node = nodes.pop()
        print(current_node.value)
        nodes.extend(current_node.children)

def __str__(self, level=0):
    ret = " " * level + str(self.value) + "\n"
    for child in self.children:
        ret += child.__str__(level + 1)
    return ret

# Create a tree
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

print("Tree structure:")
print(root)

print("\nTraversal:")
root.traverse()

```

Questions:

- 1 What is the main difference between a binary tree and a general tree?
- 2 In a Binary Search Tree, where would you find the minimum value? Where would you find the maximum value?
- 3 How does a complete binary tree differ from a full binary tree?
- 4 What tree traversal method would you use to delete a tree properly? Modify the source codes.

III. Results

```
... Tree structure:
    Root
      Child 1
        Grandchild 1
      Child 2
        Grandchild 2

    Traversal:
    Root
    Child 2
    Grandchild 2
    Child 1
    Grandchild 1
```

ANSWERS

- 1. A binary tree is a type of tree where each node can have at most two children. In contrast, general tree allows any number of children per node, depending on how the structure is defined. Because of this, binary trees follow stricter organization rules and are easier to use, while general trees offer more flexibility for representing hierarchical structures.
- 2. In a Binary Search Tree (BST), the minimum value is found by continuously moving to the left child starting from the root until no further left child exists. This works because the BST property ensures that left children always contain smaller values. Similarly, the maximum value is found by moving to the right child repeatedly until the last rightmost node is reached, which contains the largest value in the tree.
- 3. In a Binary Search Tree (BST), the minimum value is found by continuously moving to the left child starting from the root until no further left child exists. This works because the BST property ensures that left children always contain smaller values. Similarly, the maximum value is found by moving to the right child repeatedly until the last rightmost node is reached, which contains the largest value in the tree.
- 4. To delete a tree properly, a post-order traversal is preferred because it ensures that a node’s children are deleted before the node itself. This prevents issues where a parent node is deleted while its children are still in memory.

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []
```

```

def add_child(self, child_node):
    self.children.append(child_node)

def remove_child(self, child_node):
    self.children = [child for child in self.children if child != child_node]

def delete_tree(self):
    # Post-order deletion
    for child in self.children:
        child.delete_tree()
    print(f"Deleting node: {self.value}")
    self.children = []

def traverse(self):
    nodes = [self]
    while nodes:
        current_node = nodes.pop()
        print(current_node.value)
        nodes.extend(current_node.children)

def __str__(self, level=0):
    ret = " " * level + str(self.value) + "\n"
    for child in self.children:
        ret += child.__str__(level + 1)
    return ret

# Example usage
root = TreeNode("Root")
child1 = TreeNode("Child 1")
child2 = TreeNode("Child 2")
grandchild1 = TreeNode("Grandchild 1")
grandchild2 = TreeNode("Grandchild 2")

root.add_child(child1)
root.add_child(child2)
child1.add_child(grandchild1)
child2.add_child(grandchild2)

```

```
print("Deleting the entire tree:")  
root.delete_tree()
```

IV. Conclusion

Understanding tree structures and their variations—such as general trees, binary trees, and binary search trees—helps highlight how different organizational rules support different types of operations. Concepts like where minimum and maximum values are located in a BST, or how complete and full binary trees differ, show how structure affects efficiency and usage. Applying proper traversal methods, especially post-order for safe deletion, emphasizes the importance of processing nodes in the correct sequence. Overall, these concepts illustrate how trees remain a powerful and flexible data structure for organizing, searching, and managing hierarchical information in computer science.

References

- [1] Co Arthur O.. “University of Caloocan City Computer Engineering Department Honor Code,” UCC-CpE Departmental Policies, 2020.